

Research Brief: Model Serving and CI/CD in Modern Data Engineering

TL;DR

- **Model serving** is the production interface that turns a trained model into a low-latency, scalable, monitorable, and versioned prediction service; peer-reviewed work (Crankshaw et al., NSDI '17; Olston et al., NIPS-LS '17; Cox et al., ICML-W '20) frames it as the "missing piece" between ML frameworks and applications and ties it tightly to containers (Docker), orchestrators (Kubernetes) and serverless (Knative).
- **CI/CD for ML** extends DevOps with continuous training, data/model/code versioning and statistical testing, addressing the "hidden technical debt" identified by Sculley et al. (NIPS '15) and operationalised in MLOps reference architectures (Kreuzberger et al., IEEE Access 2023; Amershi et al., ICSE-SEIP '19; Karlaš et al., KDD '20).
- **Strongest peer-reviewed framework pairing:** for serving, **TensorFlow Serving** (Olston et al. 2017) + **KServe/KFServing** (Cox et al. 2020); for CI/CD, **Jenkins** + **GitHub Actions** (the latter empirically characterised for MLOps by Calefato et al., ESEM '22). MLflow's serving component (Zaharia et al., IEEE Data Eng. Bull. 2018) is an equally well-cited alternative if a lifecycle-integrated serving stack is preferred. [Google Research](#)

Key Findings

1. Role and importance of model serving

Most ML frameworks historically optimised *training*, leaving inference as an unsolved engineering problem; Crankshaw et al. explicitly motivated Clipper by observing that "most machine learning frameworks and systems only address model training and not deployment," yet production demands real-time, accurate, robust predictions under heavy load (latencies under ~100 ms, sustained throughput, graceful upgrades). Subsequent peer-reviewed work formalises four cross-cutting concerns: (i) **low-latency / high-throughput inference** via adaptive batching, caching and hardware acceleration (Crankshaw et al. 2017; Olston et al. 2017; the survey by Kojas 2023 on serverless ML inference; Li et al. 2024 on LLM serving); (ii) **model versioning and rollouts** (canary, shadow, A/B, blue-green) which Cox et al. (2020) describe as first-class features of KFServing/KServe on top of Knative; (iii) **monitoring and drift detection**, integrated in KServe via Alibi-Detect and discussed as a core MLOps principle in Kreuzberger et al. (2023) and Paleyes et al.'s ACM Computing Surveys review (2022); and (iv) **framework heterogeneity**, since organisations must serve TensorFlow, PyTorch, XGBoost, scikit-learn, and ONNX models behind a uniform REST/gRPC contract (Olston et al. 2017; Lazzeri-Albano-style benchmarks such as the 2024 arXiv evaluation of TensorFlow Serving, TorchServe, MLflow, MLServer and BentoML). Model serving is the natural anchor for **containerisation** (Docker), **orchestration** (Kubernetes — see Burns et al., ACM Queue 2016, on the

Borg→Omega→Kubernetes lineage) and **serverless** scale-to-zero patterns (Cox et al. 2020), and it is where data engineering pipelines actually deliver business value. (HackMD + 8)

2. Role and importance of CI/CD for ML

Sculley et al. (NIPS 2015) showed that ML systems accrue *hidden technical debt* through entanglement, training-serving skew, undeclared consumers, feedback loops and configuration drift — debts that ordinary software CI/CD does not detect. Amershi et al. (ICSE-SEIP 2019) and Lwakatare et al. (XP 2019) confirmed empirically that data versioning, model entanglement and reproducibility are the dominant engineering pain points in industrial ML. The community response is **MLOps**, which Kreuzberger, Köhl & Hirschl (IEEE Access 2023) define through nine principles — including CI/CD, continuous training (CT), workflow orchestration, reproducibility, versioning, and monitoring — implemented via tools such as Git, DVC, MLflow, Jenkins, GitHub Actions, GitLab CI, Argo and Tekton. Renggli et al. (SysML 2019) and Karlaš et al. (ACM SIGKDD 2020) showed why "ML as a first-class CI citizen" is non-trivial: each test reuse leaks information about the test set, so principled CI services must bound generalisation error (their *ease.ml/ci* provides single-accuracy-point tolerance with 0.999 reliability). Calefato, Lanubile & Quaranta (ESEM 2022) empirically studied MLOps practices in GitHub Actions and CML workflows and found adoption is still nascent but growing. CI/CD for ML therefore matters because, as the Paleyes et al. (2022) survey concludes, problems appear at **every** stage of deployment — data, training, integration, monitoring, updating — and only automated, reproducible pipelines can keep models valid as data and the world drift. (lcase-conferences + 10)

3. Recommended framework pairing (strongest peer-reviewed backing)

Model serving 1 — TensorFlow Serving. Described by Olston et al. (NIPS 2017 Workshop on ML Systems / arXiv:1712.06139) as a flexible, high-performance system that decouples model lifecycle (loading, versioning, traffic routing) from inference execution; achieves ~100k requests/s/core in Google production, is the default reference benchmark in the model-serving comparative evaluation by Lazzeri-Albano et al. (arXiv 2024), and integrates natively with Docker/Kubernetes. (arXiv + 3)

Model serving 2 — KServe (formerly KFServing). Documented by Cox, Sun, Tarn et al. (ICML 2020 Workshop on Challenges in Deploying and Monitoring ML Systems / arXiv:2007.07366); built on Knative serverless on Kubernetes, it provides a uniform `InferenceService` CRD that abstracts TensorFlow, PyTorch, XGBoost, scikit-learn, ONNX and Triton runtimes, with autoscaling (including scale-to-zero), GPU support, canary rollouts, payload logging and integrated drift/outlier detection — the single best peer-reviewed exemplar of the convergence of serving, containers, orchestration and serverless. (arXiv + 5)

CI/CD 1 — Jenkins. The de facto industrial CI server; its impact on continuous-integration practice is empirically characterised by an extensive body of peer-reviewed work (e.g., Soares et al.'s 2022 systematic literature review of 101 CI studies, *Journal of Systems and Software* / arXiv:2103.05451; Hilton et al., ASE 2016 on usage and benefits of CI). Through plugins it integrates with Docker, Kubernetes, MLflow and Argo CD, making it a frequent backbone of

MLOps reference architectures (Kreuzberger et al. 2023; Symeonidis et al., IEEE CCWC 2022).

arxiv + 2

CI/CD 2 — GitHub Actions. A workflow-automation, event-driven CI/CD platform whose ML-specific use is empirically investigated by Calefato, Lanubile & Quaranta ("A Preliminary Investigation of MLOps Practices in GitHub", ESEM 2022) and surveyed by Saroar & Nayebi ("Developers' Perception of GitHub Actions", arXiv 2023). It is the dominant CI/CD substrate cited in MLOps tool reviews (Kreuzberger et al. 2023; Symeonidis et al. 2022) and pairs naturally with CML and DVC for data/model versioning. Uniba

4. Broader ecosystem connections

All four frameworks are built around containers: serving systems ship the trained model as a Docker image; orchestrators (Kubernetes; Burns et al., *ACM Queue* 2016) handle replication, autoscaling and rollouts; KServe further leverages Knative for serverless scale-to-zero (Cox et al. 2020). On the CI/CD side, Jenkins and GitHub Actions both produce and promote those container images via container registries, and modern GitOps stacks frequently chain them with Argo CD or Tekton for Kubernetes-native deployment. Distributed-ML systems (e.g., Spark, Ray, Horovod) interact with this stack mainly at the *training* boundary — feeding artefacts into MLflow or DVC, which the CI pipeline then promotes to the serving layer. DigitalOcean

Details — IEEE-formatted reference list

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Montréal, Canada, 2015, pp. 2503–2511.

[2] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *Proc. 14th USENIX Symp. Networked Systems Design and Implementation (NSDI '17)*, Boston, MA, USA, Mar. 2017, pp. 613–627. ISBN 978-1-931971-37-9. USENIX

[3] C. Olston, N. Fiedel, K. Gorovoy, J. Harmsen, L. Lao, F. Li, V. Rajashekhar, S. Ramesh, and J. Soyke, "TensorFlow-Serving: Flexible, high-performance ML serving," in *Proc. NIPS 2017 Workshop on ML Systems (LearningSys)*, Long Beach, CA, USA, 2017. arXiv:1712.06139.

OpenAIRE

[4] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, Dec. 2018.

Semantic Scholar

[5] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023, doi: 10.1109/ACCESS.2023.3262138. Mendeley Mendeley

[6] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *Proc. 2019*

IEEE/ACM 41st Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP), Montréal, Canada, May 2019, pp. 291–300, doi: 10.1109/ICSE-SEIP.2019.00042. [Semantic Scholar](#)

[7] L. E. Lwakatare, A. Raj, J. Bosch, H. H. Olsson, and I. Crnkovic, "A taxonomy of software engineering challenges for machine learning systems: An empirical investigation," in *Agile Processes in Software Engineering and Extreme Programming (XP 2019)*, LNBIP vol. 355, Springer, 2019, pp. 227–243, doi: 10.1007/978-3-030-19034-7_14. [dblp](#)

[8] B. Karlaš, M. Interlandi, C. Renggli, W. Wu, C. Zhang, D. Mukunthu Iyappan Babu, J. Edwards, C. Lauren, A. Xu, and M. Weimer, "Building continuous integration services for machine learning," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD '20)*, Virtual Event, USA, Aug. 2020, pp. 2407–2416, doi: 10.1145/3394486.3403290.

[UW Computer Sciences](#)

[9] C. Renggli, B. Karlaš, B. Ding, F. Liu, K. Schawinski, W. Wu, and C. Zhang, "Continuous integration of machine learning models with ease.ml/ci: Towards a rigorous yet practical treatment," in *Proc. 2nd Conf. Systems and Machine Learning (SysML '19)*, Stanford, CA, USA, Apr. 2019. arXiv:1903.00278. [ACM Digital Library](#) [arXiv](#)

[10] C. Cox, D. Sun, E. Tarn, A. Singh, R. Kelkar, and D. Goodwin, "Serverless inferencing on Kubernetes," in *ICML 2020 Workshop on Challenges in Deploying and Monitoring Machine Learning Systems*, Vienna, Austria (virtual), Jul. 2020. arXiv:2007.07366. [arxiv](#)

[11] M. M. John, H. H. Olsson, and J. Bosch, "Towards MLOps: A framework and maturity model," in *Proc. 47th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, Palermo, Italy, Sep. 2021, pp. 334–341, doi: 10.1109/SEAA53835.2021.00050. [researchr](#) [arXiv](#)

[12] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, art. 114, pp. 1–29, Dec. 2022, doi: 10.1145/3533378. [Semantic Scholar](#)

[13] G. Symeonidis, E. Nerantzis, A. Kazakis, and G. A. Papakostas, "MLOps – Definitions, tools and challenges," in *Proc. 2022 IEEE 12th Annual Computing and Communication Workshop and Conf. (CCWC)*, Las Vegas, NV, USA, Jan. 2022, pp. 0453–0460, doi: 10.1109/CCWC54503.2022.9720902. [arXiv](#)

[14] F. Calefato, F. Lanubile, and L. Quaranta, "A preliminary investigation of MLOps practices in GitHub," in *Proc. 16th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM '22)*, Helsinki, Finland, Sep. 2022, pp. 283–288, doi: 10.1145/3544902.3546636.

[ACM Digital Library](#)

[15] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM (ACM Queue)*, vol. 59, no. 5, pp. 50–57, May 2016, doi: 10.1145/2890784.

[16] E. Soares, G. Sizilio, J. Santos, D. A. da Costa, and U. Kulesza, "The effects of continuous integration on software development: A systematic literature review," *Empirical Software Engineering*, vol. 27, no. 3, art. 78, 2022 (preprint arXiv:2103.05451).

(Optional supplementary, for depth on serving benchmarks: K. Kojs, "A survey of serverless machine learning model inference," *arXiv:2311.13587*, Nov. 2023; and the comparative serving-framework benchmark "On the cost of model-serving frameworks: An experimental evaluation," *arXiv:2411.10337*, 2024 — both useful for justifying the TF-Serving / KServe choice on empirical grounds.) [Semantic Scholar](#)

Suggested seminar discussion questions

1. **Where does CI for ML actually break traditional CI semantics?** Karlaš et al. and Renggli et al. argue that test reuse leaks information about the test set, so each "build" silently overfits validation. How should a Data Engineering team design CI gates (Jenkins/GitHub Actions pipelines) so that statistical generalisation guarantees survive frequent commits, and is [ease.ml/ci](#)-style probabilistic testing realistic in industry? [UW Computer Sciences](#)
2. **Is serverless serving (KServe/Knative) genuinely cheaper for production ML, or is it mostly suitable for "long-tail" models?** Cox et al. (2020) and Kojs (2023) discuss cold-start, GPU autoscaling and SLO trade-offs — when would you prefer a dedicated TensorFlow Serving deployment over a scale-to-zero KServe [InferenceService](#), and how should that decision be encoded in the CI/CD pipeline that promotes a model from staging to production?
3. **MLOps maturity vs. organisational reality.** Kreuzberger et al. (2023) and John, Olsson & Bosch (2021) propose maturity models in which CI/CD/CT, monitoring and feedback loops are fully automated. Calefato et al. (2022) found that, in open-source GitHub projects, MLOps automation is still rare. What are the minimum CI/CD and serving capabilities a small data-engineering team should adopt to avoid Sculley et al.'s "hidden technical debt" without paying full enterprise tooling cost — and how does a Flask + Celery + RabbitMQ deployment on an OpenStack VM map onto (or fall short of) that minimum?

[ACM Digital Library](#)

Caveats

- Several foundational works are *workshop* papers rather than full peer-reviewed conference papers: Olston et al. (NIPS 2017 LearningSys) and Cox et al. (ICML 2020 Workshop on Challenges in Deploying and Monitoring ML) are widely cited but are workshop-track contributions; the corresponding arXiv preprints (1712.06139, 2007.07366) are the most-cited form. Zaharia et al. 2018 appeared in the *IEEE Data Engineering Bulletin*, which is editorially curated rather than blind peer-reviewed in the strict sense — for a strictly peer-reviewed MLflow citation, the follow-up *Developments in MLflow* (DEEM '20 / SIGMOD workshop) by Chen et al. is a stronger choice. [Docslib + 2](#)
- The Kreuzberger et al. paper is sometimes cited with year 2022 (acceptance) and 2023 (publication, IEEE Access vol. 11, pp. 31866–31879) — the 2023 form is the canonical one to cite.

- "Hidden Technical Debt" (Sculley et al.) is canonically cited as NIPS 2015; some references list pp. 2503–2511 (NeurIPS proceedings volume 28).
- BentoML, TorchServe and NVIDIA Triton are well-supported by *industry* literature and recent comparative benchmarks (e.g., the 2024 arXiv "On the cost of model-serving frameworks" study) but lack a single foundational peer-reviewed paper; for a master's seminar requiring two peer-reviewed serving frameworks, **TensorFlow Serving + KServe** is the most defensible pairing. If the student prefers a lifecycle-centric pairing, **TensorFlow Serving + MLflow Models** also has solid peer-reviewed grounding via [3] and [4].
- The ICML 2020 workshop paper for KServe predates the project's rename from KFServing; both names refer to the same system and are interchangeable in citation, but the system name should be explained on first mention in the seminar answer. (Alibaba Cloud)
- For the student's practical anecdote (Flask REST API + TensorFlow/Keras + Celery + RabbitMQ on an OpenStack VM): this is a textbook example of the "naïve" serving architecture that Olston et al. (2017) and BentoML's own writings critique — single-process Python with WSGI cannot exploit dynamic batching, multi-version routing or scale-to-zero. Mentioning this honestly and contrasting it with what TensorFlow Serving or KServe would have provided (gRPC, batching, canary, autoscaling, drift detection) is exactly the kind of reflection a Data Engineering seminar rewards.